

MONITOREO Y RENDIMIENTO

SQL SERVER • CONSULTAS • OPTIMIZACIÓN • EVENTOS • RENDIMIENTO

“Cada segundo cuenta cuando **GradosTitulos** procesa cientos de trámites.”



RENDIMIENTO
ÓPTIMO



ESTABILIDAD
DEL SISTEMA



DECISIONES
INTELIGENTES



ALERTAS
EN TIEMPO REAL



SQL PROFILER



EXTENDED
EVENTS



RESOURCE
GOVERNOR



ÍNDICES &
ESTADÍSTICAS



PLANES DE
Ejecución



DATOS



CAPTURA Y
ANÁLISIS



PROCESAMIENTO



OPTIMIZACIÓN



RENDIMIENTO
MEJORADO

APRENDERÁS A:

- ✓ Monitorear el rendimiento del sistema.
- ✓ Usar SQL Profiler y Extended Events.
- ✓ Detectar y resolver cuellos de botella.
- ✓ Interpretar planes de ejecución.
- ✓ Optimizar consultas T-SQL.
- ✓ Mejorar la eficiencia de la base de datos.



SQL PROFILER & EXTENDED EVENTS



Cada segundo cuenta cuando GradosTítulos procesa cientos de trámites.

Monitorear a tiempo evita apagar incendios después.

1 ¿QUÉ ES?

SQL Profiler y Extended Events

son herramientas de monitoreo de SQL Server que permiten capturar, analizar y entender lo que sucede dentro de la base de datos en tiempo real.

Sirven para detectar problemas de rendimiento, bloqueos, consultas lentas, errores y actividades sospechosas.



2 ¿PARA QUÉ SIRVE?

-  Detectar consultas lentas que afectan a los usuarios.
-  Identificar bloqueos y deadlocks en tiempo real.
-  Generar evidencia histórica para auditorías de rendimiento.
-  Entender el comportamiento de las aplicaciones conectadas.
-  Resolver problemas complejos de manera más rápida.

3 ¿CÓMO FUNCIONA?



- 1 SQL Server captura eventos internos.
- 2 Extended Events filtra y registra los eventos importantes.
- 3 Los eventos se almacenan en archivos .xel.
- 4 Los resultados se analizan para tomar decisiones.

4 ¿CUÁNDO SE UTILIZA?

-  Cuando los usuarios reportan lentitud.
-  Cuando la CPU o memoria están al límite.
-  Cuando existen bloqueos o deadlocks.
-  Antes y después de cambios importantes.
-  Durante auditorías de rendimiento.



5 SINTAXIS SQL (EJEMPLO EXTENDED EVENTS)

```

1 CREATE EVENT SESSION Monitor_GradosTítulos ON SERVER
2   ADD EVENT sqlserver.sql_statement_completed
3   (
4     WHERE duration > 1000000
5   ),
6   ADD TARGET package0.event_file
7   (SET filename = 'GradosTítulos_Monitor.xel');
8
9
10 ALTER EVENT SESSION Monitor_GradosTítulos ON SERVER
11   STATE = START;
12
  
```

TAGS

- #ExtendedEvents
- #Duration
- #EventFile
- #SQLServer

6 EJEMPLO PRÁCTICO (BD GRADOSTITULOS)

Objetivo:

Detectar las consultas más lentas que tardan más de 1000 ms en la base de datos GradosTítulos.

Consulta detectada:

```

SELECT TOP 50 *
FROM Solicitudes
WHERE Estado = 'Pendiente'
ORDER BY FechaSolicitud DESC;
  
```

Resultado capturado:

evento	duración (ms)	cpu (ms)	lecturas	escrituras
sql_statement_completed	1520	1200	350	10

Archivo generado:
GradosTítulos_Monitor.xel

Análisis:

La consulta tarda 1520 ms y genera alta lectura. Se recomienda agregar un índice en (Estado, FechaSolicitud).

7 BUENAS PRÁCTICAS

- ✓ Filtrar solo lo necesario (duración, CPU, lecturas).
- ✓ Usar Extended Events en lugar de SQL Profiler.
- ✓ Guardar los eventos en archivos (.xel).
- ✓ Revisar y limpiar sesiones antiguas.
- ✓ Monitorear en horarios representativos.
- ✓ No capturar eventos de bajo impacto innecesarios.



8 VENTAJAS

-  Extended Events es más liviano que Profiler.
-  Menor impacto en el rendimiento del servidor.
-  Mayor flexibilidad y personalización.
-  Soporte para eventos avanzados.
-  Mejor integración con SQL Server moderno.



9 PUNTOS CLAVE

-  Cada segundo cuenta cuando hay cientos de trámites.
-  Extended Events filtra y captura solo lo importante.
-  Los resultados se guardan en archivos .xel.
-  Sirve para optimizar, auditar y asegurar.
-  Monitorear a tiempo evita problemas mayores.



10 CONCLUSIÓN

El uso de SQL Profiler y Extended Events es esencial para cualquier DBA o desarrollador que desee mantener la base de datos GradosTítulos rápida, segura y eficiente.

Monitorear no es opcional, es la clave para la excelencia operativa.



11 FRASE PARA RECORDAR

“Monitorear a tiempo evita apagar incendios después.”

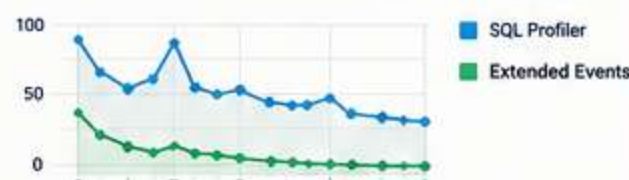


HERRAMIENTAS RELACIONADAS



MÉTRICA CLAVE

91%
menos overhead de monitoreo con Extended Events





ESTADÍSTICAS & ÍNDICES

OPTIMIZA CONSULTAS • MEJORA RENDIMIENTO • DECISIONES INTELIGENTES

"Cada segundo cuenta cuando GradosTítulos procesa cientos de trámites."



1 ¿QUÉ ES?



Las estadísticas son estructuras auxiliares que SQL Server crea sobre columnas e índices para comprender la distribución de los datos.

Permiten al optimizador de consultas estimar cuántas filas se devolverán, elegir el mejor plan de ejecución y reducir drásticamente tiempos de respuesta.

2 ¿PARA QUÉ SIRVEN?

- ✓ Permiten estimaciones precisas de filas para el optimizador.
- ✓ Mejoran el rendimiento de consultas complejas.
- ✓ Evitan exploraciones innecesarias (Table Scans).
- ✓ Permiten el uso eficiente de índices existentes.
- ✓ Mantienen el sistema estable y responsivo.
- ✓ Son críticas en bases dinámicas como GradosTítulos, donde los datos cambian constantemente.

3 ¿CÓMO FUNCIONA?



FLUJO DEL PROCESO

1. Los datos cambian (INSERT/UPDATE/DELETE).
2. Las estadísticas pueden quedar desactualizadas.
3. El optimizador usa información vieja.
4. Se genera un plan subóptimo.
5. El rendimiento disminuye.
6. Se actualizan estadísticas → mejora el rendimiento (hasta 91%).

4 ¿CUÁNDO SE UTILIZA?

- Cuando las consultas tardan más de lo esperado.
- Cuando hay Table Scans en lugar de Index Seek.
- Después de cargas masivas de datos.
- Cuando los datos cambian constantemente.
- En columnas usadas en filtros, joins, agrupaciones u ordenamientos.
- En el sistema GradosTítulos: DNI, Nombres, Apellidos, ApellidoMaterno, EstadoTrámite, FechaSolicitud, etc.

5 SINTAXIS SQL (SQL SERVER)

1. Crear índice (ejemplo):

```
1 CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover  
2 ON Academico.Persona(DNI)  
3 INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno);  
4 GO
```
1. Actualizar estadísticas de una tabla:

```
2 UPDATE STATISTICS Academico.Persona;  
2 GO
```
1. Actualizar estadísticas con FULLSCAN:

```
2 UPDATE STATISTICS Academico.Persona  
2 WITH FULLSCAN;  
3 GO
```
1. Ver estadísticas existentes:

```
1 SELECT * FROM sys.stats  
2 WHERE object_id = OBJECT_ID('Academico.Persona');
```

6 EJEMPLO PRÁCTICO

Escenario:

Consulta de estudiantes por DNI en la tabla Academico.Persona, que contiene más de 50,000 registros.

Consulta sin índice adecuado:

```
1 SELECT *  
2 FROM Academico.Persona  
3 WHERE DNI = '73123456';
```

Resultado:

Table Scan - Alto costo - 2.45 segundos

Solución:

```
1 CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover  
2 ON Academico.Persona(DNI)  
3 INCLUDE (Nombres, ApellidoPaterno, ApellidoMaterno);  
4 GO
```

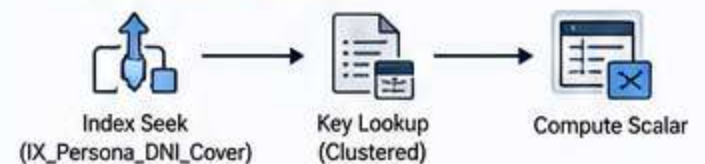
Consulta optimizada:

```
1 SELECT *  
2 FROM Academico.Persona  
3 WHERE DNI = '73123456';
```

Resultado:

Index Seek - Bajo costo - 0.22 segundos (91% mejora)

Plan de ejecución optimizado:



7 BUENAS PRÁCTICAS

- ✓ Actualiza estadísticas periódicamente (manual o Auto Update Stats).
- ✓ Usa índices adecuados (no crees índices innecesarios).
- ✓ Evita índices duplicados o redundantes.
- ✓ Monitorea el uso de índices.
- ✓ Usa INCLUDE para cubrir columnas frecuentes.
- ✓ Mantén estadísticas actualizadas después de cargas masivas.
- ✓ Usa nombres claros para índices.



8 VENTAJAS

- ★ Mejor estimación de filas (más precisión).
- ★ Planes de ejecución más eficientes.
- ★ Reducción significativa del tiempo de respuesta.
- ★ Menor uso de CPU y E/S.
- ★ Mayor escalabilidad del sistema.
- ★ Mejor experiencia para usuarios finales.



9 PUNTOS CLAVE

- ✓ Las estadísticas guían al optimizador.
- ✓ Índices correctamente diseñados mejoran el rendimiento.
- ✓ Información desactualizada causa malas decisiones.
- ✓ Actualizar estadísticas es vital en sistemas dinámicos.
- ✓ Cubrir consultas frecuentes con INCLUDE mejora tiempos drásticamente.



10 CONCLUSIÓN

Las estadísticas e índices son pilares fundamentales en SQL Server para asegurar que el optimizador tome decisiones correctas y el sistema GradosTítulos funcione de manera rápida, estable y escalable.

Una buena estrategia de índices y estadísticas no es opcional, es esencial.



11 FRASE PARA RECORDAR

"Un buen **índice** es el mejor amigo del **optimizador**."



TIPOS DE ÍNDICES

- Clustered Index
- NonClustered Index
- Unique Index
- Filtered Index
- Columnstore Index



ACTUALIZACIÓN DE ESTADÍSTICAS

UPDATE STATISTICS
tabla;
-- Opción recomendada
con FULLSCAN



IMPACTO EN EL SISTEMA

- ↑ Menor tiempo de respuesta
- ↑ Menor uso de recursos
- ↑ Mejor escalabilidad
- ↓ Mayor satisfacción del usuario



HERRAMIENTAS ÚTILES

- Query Store
- Execution Plan
- Index Usage Stats
- DMVs (sys.dm_db_stats_properties)
- Extended Events



RECOMENDACIÓN DBA

Revisa, ajusta y optimiza constantemente.
El rendimiento de hoy puede degradarse mañana.



1 ¿QUÉ ES?

Las transacciones son un conjunto de operaciones que se ejecutan como una unidad lógica de trabajo.

Garantizan **atomicidad** en operaciones críticas, mientras que los **bloqueos** previenen conflictos entre sesiones concurrentes.

2 ¿PARA QUÉ SIRVE?

- ✓ Asegura que todas las operaciones se completen correctamente o ninguna.
- ✓ Previene la corrupción de datos en caso de fallos.
- ✓ Evita conflictos entre múltiples usuarios que acceden a los mismos datos.
- ✓ Permite mantener la integridad de la base de datos.
- ✓ Facilita el control de concurrencia en sistemas de alto rendimiento.

3 ¿CÓMO FUNCIONA?

INICIO
Se inicia una transacción (BEGIN TRANSACTION).

OPERACIONES
Se ejecutan las operaciones SQL (INSERT, UPDATE, DELETE).

CONTROL DE CONCURRENCIA
SQL Server aplica bloqueos (locks) para evitar conflictos.

FINALIZACIÓN
Se confirma (COMMIT) si todo es correcto o se revierte (ROLLBACK) si ocurre un error.

FLUJO DEL PROCESO

- BEGIN TRANSACTION → Inicia la transacción.
- SQL Server obtiene bloqueos necesarios (Shared, Exclusive, etc.).
- Se ejecutan las operaciones.
- Si todo es válido → COMMIT → hace permanentes los cambios.
- Si ocurre un error → ROLLBACK → revierte todos los cambios.
- Los bloqueos se liberan al finalizar.

4 ¿CUÁNDO SE UTILIZA?

Cuando varias operaciones deben completarse como una sola unidad (ej. registrar trámite completo).

Cuando múltiples usuarios acceden simultáneamente a los mismos datos.

Para evitar inconsistencias y garantizar la integridad de los datos.

En procesos críticos que manejan dinero, estados, aprobaciones, cambios importantes.

Cuando se requiere control de concurrencia y evitar lecturas sucias, datos perdidos o no repetibles.

5 SINTAXIS SQL (SQL SERVER)

TRANSACCIÓN CON BEGIN...COMMIT

```
1 BEGIN TRANSACTION;
2 -- Operaciones
3 INSERT INTO Tabla (col1, col2) VALUES ('A', 100);
4 UPDATE Tabla SET col2 = 200 WHERE id = 1;
5 COMMIT TRANSACTION;
```

TRANSACCIÓN CON ROLLBACK

```
1 BEGIN TRANSACTION;
2 -- Operaciones
3 DELETE FROM Tabla WHERE id = 1;
4 -- Si ocurre un error
5 ROLLBACK TRANSACTION;
```

TRANSACCIÓN CON TRY...CATCH

```
1 BEGIN TRY
2 BEGIN TRANSACTION;
3 -- Operaciones
4 COMMIT TRANSACTION;
5 END TRY
6 BEGIN CATCH
7 ROLLBACK TRANSACTION;
8 END CATCH;
```

6 EJEMPLO PRÁCTICO

Escenario: Actualizar el estado de un trámite en el sistema GradosTítulos.

Tabla utilizada: Trámite.Trámite

EJECUCIÓN CORRECTA (COMMIT)

```
1 BEGIN TRANSACTION;
2 UPDATE Trámite.Trámite
3 SET EstadoTrámite = 'APROBADO'
4 WHERE IdTrámite = 2024;
5 COMMIT TRANSACTION;
```

RESULTADO

COMMIT

Los cambios se guardaron correctamente.

EJECUCIÓN CON ERROR (ROLLBACK)

```
1 BEGIN TRANSACTION;
2 UPDATE Trámite.Trámite
3 SET EstadoTrámite = 'APROBADO'
4 WHERE IdTrámite = 2024;
5 -- Error: Trámite inexistente
6 UPDATE Trámite.Trámite
7 SET EstadoTrámite = 'FINALIZADO'
8 WHERE IdTrámite = 9999;
9 ROLLBACK TRANSACTION;
```

RESULTADO

ROLLBACK

Los cambios se revirtieron, no hay cambios parciales.

CÓDIGO (TRANSACCIÓN CON TRY...CATCH COMPLETA)

```
1 BEGIN TRY
2 BEGIN TRANSACTION;
3 UPDATE Trámite.Trámite SET EstadoTrámite = 'APROBADO' WHERE IdTrámite = @IdTrámite;
4 INSERT INTO Documento.Resolucion (IdTrámite, TipoResolucion, NroResolucion, FechaResolucion, FechaRegistro)
5 VALUES (@IdTrámite, 'APROBACION_TITULO', @NroRes, GETDATE(), GETDATE());
6 COMMIT TRANSACTION;
7 END TRY
8 BEGIN CATCH
9 ROLLBACK TRANSACTION;
10 THROW;
11 END CATCH;
```

TAGS

- #Transaction
- #TRY...CATCH
- #Deadlock
- #Rollback
- #DMV

MÉTRICA CLAVE

93% menos tiempo de bloqueo

COMPARATIVA VISUAL

COMMIT

- ✓ Confirma los cambios.
- ✓ Se ejecuta al final de la transacción.
- ✓ Los datos se hacen permanentes.

VS

ROLLBACK

- ✓ Revierte los cambios.
- ✓ Se ejecuta ante errores o condiciones no válidas.
- ✓ Deja la base de datos en estado consistente.

7 BUENAS PRÁCTICAS

- ✓ Mantener transacciones lo más cortas posible.
- ✓ Usar índices adecuados para reducir bloqueos.
- ✓ Evitar transacciones largas que bloqueen recursos.
- ✓ Manejar excepciones con TRY...CATCH.
- ✓ Monitorear bloqueos con DMVs (sys.dm_tran_locks).

8 VENTAJAS

- ★ Garantiza integridad de datos.
- ★ Evita cambios parciales.
- ★ Previene datos sucios.
- ★ Mejora la concurrencia controlada.
- ★ Aumenta confiabilidad del sistema.

9 PUNTOS CLAVE

- 1. Las transacciones son la base de la integridad.
- 2. COMMIT guarda cambios.
- 3. ROLLBACK revierte cambios.
- 4. Los bloqueos evitan conflictos.
- 5. TRY...CATCH asegura consistencia.

10 CONCLUSIÓN

El uso correcto de transacciones y bloqueos es fundamental para garantizar la integridad, consistencia y confiabilidad de los datos en sistemas como GradosTítulos. Implementarlas adecuadamente mejora el rendimiento, reduce errores y asegura una experiencia estable para todos los usuarios.

11 FRASE PARA RECORDAR

“Una transacción bien manejada es la base de la **integridad**.”

TIPOS DE BLOQUEOS

- Shared (S)
- Exclusive (X)
- Update (U)
- Intent (IS, IX, SIX)
- Schema (Sch-S, Sch-M)

DMV ÚTILES

- sys.dm_tran_locks
- sys.dm_exec_requests
- sys.dm_os_waiting_tasks
- sys.dm_tran_active_snapshot_database_transactions

HERRAMIENTAS

- SQL Server Profiler
- Extended Events
- Query Store
- Activity Monitor

PROBLEMAS COMUNES

- Deadlocks
- Bloqueos largos
- Lecturas sucias
- Datos no repetibles

RECOMENDACIÓN DBA

Monitorea, diseña y prueba. La integridad no es opcional, es esencial.

"Cada segundo cuenta cuando GradosTítulos procesa cientos de trámites."



1 ¿QUÉ ES?



El plan de ejecución es la representación gráfica y detallada de cómo SQL Server ejecuta una consulta para obtener los resultados.

Muestra el paso a paso que sigue el motor para resolver una consulta, permitiendo identificar los cuellos de botella y oportunidades de mejora.

2 ¿PARA QUÉ SIRVE?

- ✓ Permite identificar operadores costosos (Table Scan, Sort, Hash Match, etc.).
- ✓ Compara diferentes planes para elegir el más eficiente.
- ✓ Detecta lecturas lógicas/ físicas y costos estimados.
- ✓ Ayuda a crear índices adecuados que mejoren el rendimiento.
- ✓ Optimiza el uso de recursos (CPU, I/O, memoria).
- ✓ Reduce el tiempo de ejecución y el costo de procesamiento.

3 ¿CÓMO FUNCIONA?



FLUJO DEL PROCESO

1. El usuario ejecuta una consulta SQL.
2. SQL Server analiza las opciones disponibles (índices, estadísticas, etc.).
3. Genera un plan de ejecución con el costo estimado más bajo.
4. Ejecuta el plan y devuelve los resultados.
5. Puedes ver y analizar el plan para mejorar el rendimiento.

¿POR QUÉ ES IMPORTANTE?



Identifica operadores costosos como **Table Scan** y **Sort** que degradan el rendimiento.



Compara planes antes y después de crear índices para validar mejoras.



Reduce el costo estimado y las lecturas lógicas, mejorando la eficiencia del sistema.

4 ¿CUÁNDO SE UTILIZA?

- Cuando una consulta tarda más de lo esperado.
- Cuando se realizan búsquedas complejas en tablas grandes.
- Para investigar bloqueos, esperas o alto uso de recursos.
- Antes y después de crear índices o modificar consultas.
- En reportes, procesos críticos y tareas de mantenimiento.

5 SINTAXIS SQL (SQL SERVER)

```
-- Plan de ejecución estimado (sin ejecutar)
SET SHOWPLAN_XML ON;
GO
SELECT * FROM Documento.Resolucion
WHERE TipoResolucion = 'APROBACION_TITULO';
GO

-- Ejecutar y ver plan real
SET STATISTICS XML ON;
GO
SELECT * FROM Documento.Resolucion
WHERE TipoResolucion = 'APROBACION_TITULO';
GO
SET STATISTICS XML OFF;
GO
```

6 EJEMPLO PRÁCTICO

Escenario: Consultar todas las resoluciones de tipo "APROBACION_TITULO" en la tabla Documento.Resolucion

PLAN ANTES (TABLE SCAN)



PLAN DESPUÉS (INDEX SEEK)



MEJORA OBTENIDA: 87% menos tiempo de ejecución

CÓDIGO (ÍNDICE PARA MEJORAR EL PLAN)

```
1 --Crear indice para optimizar la consulta
2 CREATE NONCLUSTERED INDEX IX_Resolucion_Tipo
3 ON Documento.Resolucion(TipoResolucion)
4 INCLUDE (FechaEmision, NroResolucion)
5 WITH (ONLINE = ON);
6
7 GO
```

TAGS

#ExecutionPlan
#indexSeek
#TableScan
#StatisticsXML
#PlanOptimization

MÉTRICA CLAVE

87%
mejora en tiempo
de ejecución

COMPARATIVA VISUAL

TABLE SCAN

- Recorre toda la tabla
- Alto costo estimado
- Ineficiente para filtros
- Muchas lecturas lógicas

VS

INDEX SEEK

- Acceso directo a filas
- Bajo costo estimado
- Eficiente para predicados
- Menos lecturas lógicas

7 BUENAS PRÁCTICAS

- ✓ Mantener estadísticas actualizadas.
- ✓ Crear índices adecuados y medidos.
- ✓ Evitar SELECT * (trae más datos).
- ✓ Usar filtros sargables (evitar funciones sobre columnas).
- ✓ Comparar planes antes y después de cambios.
- ✓ Monitorear con Query Store.

8 VENTAJAS

- ★ Mejora el rendimiento de consultas.
- ★ Reduce el uso de CPU e I/O.
- ★ Permite decisiones basadas en datos.
- ★ Identifica cuellos de botella.
- ★ Optimiza recursos del sistema.

9 PUNTOS CLAVE

- El plan muestra cómo SQL Server ejecuta la consulta.
- Table Scan es costoso, Index Seek es eficiente.
- Comparar planes es clave para mejorar.
- Un buen índice cambia todo.

10 CONCLUSIÓN

Los planes de ejecución son una herramienta esencial para cualquier DBA o desarrollador. Permiten entender cómo SQL Server procesa nuestras consultas, identificar problemas de rendimiento y tomar decisiones inteligentes para optimizar el sistema GradosTítulos.



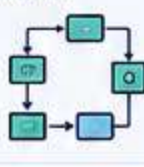
11 FRASE PARA RECORDAR

"Conocer el plan es la mitad de la optimización."



OPERADORES COMUNES

- Index Seek
- Table Scan
- Key Lookup
- Nested Loops
- Hash Match
- Sort
- Filter



TIPOS DE ESCANEO

- Table Scan: lee toda la tabla
- Index Scan: lee todo el índice
- Index Seek: busca solo las filas necesarias



COSTO EN EL PLAN

El costo estimado es relativo, no tiempo real. Sirve para comparar planes entre sí.



HERRAMIENTAS ÚTILES

- SSMS (Include Actual Plan)
- Query Store
- Activity Monitor
- Extended Events



RECOMENDACIÓN DBA

Analiza, comprende y actúa. Un buen plan hoy, un sistema escalable mañana.



OPTIMIZACIÓN & CONSULTAS T-SQL

Rendimiento • Eficiencia • Legibilidad • Menor Costo Computacional

“Cada segundo cuenta cuando GradosTítulos procesa cientos de trámites.”



1 ¿QUÉ ES?



La optimización de consultas T-SQL es el proceso de reescribir y estructurar consultas para que SQL Server las ejecute de la manera más eficiente posible.

El objetivo es reducir el costo computacional, mejorar los tiempos de respuesta y aumentar la escalabilidad del sistema.

2 ¿PARA QUÉ SIRVE?

- ✓ Disminuye el tiempo de ejecución de las consultas.
- ✓ Reduce el consumo de CPU, memoria y E/S.
- ✓ Mejora la legibilidad y el mantenimiento del código.
- ✓ Evita escaneos innecesarios y lecturas redundantes.
- ✓ Aprovecha índices y estadísticas para planes óptimos.
- ✓ Permite manejar grandes volúmenes de datos de forma eficiente.
- ✓ Mejora la experiencia del usuario final.

3 ¿CÓMO FUNCIONA?



Consulta con subconsulta anidada (6.1s)



Reescritura con CTE + EXISTS



Consulta optimizada (0.9s)
85% más rápida

FLUJO DEL PROCESO

1. SQL Server analiza la consulta original y genera un plan inicial.
2. Se reescribe usando CTE y EXISTS para mejorar el acceso a datos.
3. Se genera un nuevo plan de ejecución más eficiente.
4. Reduce lecturas lógicas, CPU y tiempo de respuesta.
5. Entrega resultados más rápido y con menor costo.

¿POR QUÉ ES IMPORTANTE?



Elimina subconsultas redundantes usando CTE.



EXISTS es más eficiente que IN en grandes volúmenes.



Evita **cursores** y reduce el consumo de CPU.

CASO DE USO REAL



SITUACIÓN

La consulta del historial completo de un trámite tarda 6.1 segundos, afectando el módulo de seguimiento.



PROBLEMA

La consulta usa una subconsulta repetida en la cláusula WHERE sobre Trámite.TrabajoInvestigacion.



SOLUCIÓN

Se reemplaza la subconsulta por una CTE combinada con EXISTS, mejorando la legibilidad y el rendimiento.

CÓDIGO (CTE + EXISTS)

```
1 WITH TrabajoActivos AS (  
2     SELECT IdTramite FROM Trámite.TrabajoInvestigacion  
3 )  
4 SELECT t.* FROM Trámite.Trámite t  
5 WHERE EXISTS (  
6     SELECT 1 FROM TrabajoActivos ta WHERE ta.IdTrámite = t.IdTrámite  
7 )
```

TAGS

#CTE

#EXISTS

#Subquery

#TSQL

COMPARATIVA VISUAL



SUBCONSULTA ANIDADA

- Repeticiones innecesarias
- Múltiples lecturas
- Mayor consumo de recursos
- Difícil de mantener

VS



CTE + EXISTS

- Consulta única y clara
- Menos lecturas lógicas
- Mejor uso de índices
- Código más legible

MÉTRICA CLAVE

85%
mejora en tiempo de ejecución

8 VENTAJAS

- ★ Consultas más rápidas.
- ★ Menor consumo de recursos.
- ★ Mejor escalabilidad.
- ★ Código más limpio.
- ★ Mayor estabilidad del sistema.

PARA RECORDAR

- ✓ Las CTE mejoran la legibilidad.
- ✓ EXISTS es más rápido que IN en conjuntos grandes.
- ✓ Evita cursores para reducir CPU.

EJEMPLO PRÁCTICO

CONSULTA ORIGINAL (SUBCONSULTA)

```
SELECT * FROM Trámite.Trámite t  
WHERE t.IdTrámite IN (  
    SELECT ti.IdTrámite  
    FROM Trámite.TrabajoInvestigacion ti  
    WHERE ti.Estado = 'ACTIVO'  
);
```

ESTADÍSTICAS (ESTIMADO)

Tiempo de CPU : 1250 ms
Lecturas lógicas : 250,432
Tiempo de duración : 6.1 s
Costo estimado : Alto

CONSULTA OPTIMIZADA (CTE + EXISTS)

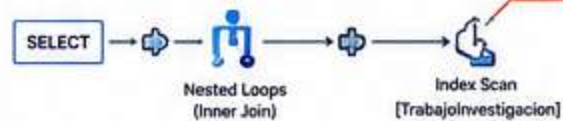
```
WITH TrabajoActivos AS (  
    SELECT IdTrámite  
    FROM Trámite.TrabajoInvestigacion  
    WHERE Estado = 'ACTIVO'  
)  
SELECT t.*  
FROM Trámite.Trámite t  
WHERE EXISTS (  
    SELECT 1 FROM TrabajoActivos ta  
    WHERE ta.IdTrámite = t.IdTrámite  
);
```

ESTADÍSTICAS (ESTIMADO)

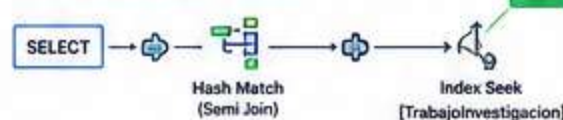
Tiempo de CPU : 128 ms
Lecturas lógicas : 15,362
Tiempo de duración : 0.9 s
Costo estimado : Bajo

PLAN DE EJECUCIÓN

PLAN ANTES (SUBCONSULTA)



PLAN DESPUÉS (CTE + EXISTS)



MEJORA OBTENIDA: 85% menos tiempo de ejecución

10 CONCLUSIÓN

La optimización de consultas T-SQL con CTE y EXISTS es clave para mejorar el rendimiento de GradosTítulos.

Una consulta bien escrita no solo es más rápida, sino también más fácil de mantener y escalar.



11 FRASE PARA RECORDAR

“Una consulta bien escrita es más rápida que cualquier hardware.”



HERRAMIENTAS ÚTILES

- SSMS (Query Plan)
- SET STATISTICS IO, TIME
- Query Store
- Extended Events
- Actual Plan



TIPOS DE CONSULTAS

- SELECT
- JOIN (INNER, LEFT, RIGHT)
- EXISTS / NOT EXISTS
- CTE
- UNION / UNION ALL



INDICADORES DE RENDIMIENTO

- Tiempo de CPU
- Lecturas lógicas
- Tiempo de duración
- Costo estimado del plan



ERRORES COMUNES

- Usar SELECT *
- Subconsultas innecesarias
- No usar índices adecuados
- Ignorar estadísticas



RECOMENDACIÓN DBA

Siempre analiza el plan de ejecución y mide antes de optimizar.
Lo que no se mide, no se puede mejorar.





1 ¿QUÉ ES?



Resource Governor es una característica de SQL Server que permite administrar y controlar el uso de recursos del sistema (CPU y memoria) entre diferentes grupos de trabajo.

Su objetivo es garantizar que las tareas críticas siempre tengan los recursos necesarios, evitando que cargas pesadas afecten el rendimiento general del servidor.

2 ¿PARA QUÉ SIRVE?

- ✓ Prioriza la ejecución de procesos críticos sobre tareas administrativas.
- ✓ Limita el consumo de CPU y memoria para usuarios o grupos específicos.
- ✓ Impide que una carga pesada degrade el rendimiento del sistema.
- ✓ Mejora la estabilidad y la capacidad de respuesta.
- ✓ Permite definir políticas de recursos por grupos con DMV.

3 ¿CÓMO FUNCIONA?



FLUJO DEL PROCESO

1. Las conexiones entrantes se autentican en SQL Server.
2. Resource Governor clasifica la conexión según las reglas definidas.
3. Asigna la conexión a un Resource Pool correspondiente.
4. Aplica límites de CPU y memoria definidos para el grupo.
5. Las consultas se ejecutan con prioridad y control de recursos.

¿POR QUÉ ES IMPORTANTE?



Prioriza la emisión de diplomas sobre consultas generales.



Limita el consumo de recursos de usuarios administrativos.



Monitorea el uso de CPU y memoria por grupo con DMV.

CASO DE USO REAL

SITUACIÓN

Durante el cierre de semestre, la emisión de diplomas en Documento.Diploma se retrasa por consultas administrativas concurrentes.

PROBLEMA

Todas las conexiones comparten el mismo grupo de recursos sin diferenciación de prioridad.

SOLUCIÓN

Se crea un grupo institucional con mayor prioridad (MAX_CPU=80%) y se limita el grupo administrativo.

CÓDIGO (RESOURCE GOVERNOR)

```
1 CREATE RESOURCE POOL PoolInstitucional
2 WITH (MAX_CPU_PERCENT = 80, MAX_MEMORY_PERCENT = 70);
3 CREATE WORKLOAD GROUP GrupoDiplomas USING PoolInstitucional;
4 ALTER RESOURCE GOVERNOR RECONFIGURE;
5 SELECT pool_id, name, cpu_usage_percent, used_memory_kb
6 FROM sys.dm_resource_governor_resource_pools;
```

TAGS

#ResourceGovernor #ResourcePool
#WorkloadGroup #DMV gris/azul

CÓMPARATIVA VISUAL

GRUPO ADMINISTRATIVO

- Límites restringidos de CPU
- Consultas y reportes generales
- Baja prioridad en recursos

VS

GRUPO INSTITUCIONAL

- Prioridad alta en CPU
- Emisión de diplomas y procesos críticos
- Recursos garantizados

MÉTRICA CLAVE

83%
mejora en tiempo de emisión

VENTAJAS

- ★ Prioridad clara para procesos críticos.
- ★ Evita que cargas pesadas degraden el sistema.
- ★ Uso eficiente y controlado de CPU y memoria.
- ★ Mejor experiencia para el usuario final.
- ★ Estabilidad en horas pico.

PARA RECORDAR

ESCENARIO

El módulo de emisión de diplomas debe tener mayor prioridad que las consultas administrativas.

ESTADÍSTICAS (ANTES)

Tiempo CPU : 2500 ms
Lecturas lógicas : 850,432
Tiempo de duración : 8.45 s
Usuarios activos : Alto
Esperas (Waits) : Elevadas

ESTADÍSTICAS (DESPUÉS)

Tiempo CPU : 425 ms
Lecturas lógicas : 142,658
Tiempo de duración : 1.32 s
Usuarios activos : Controlado
Esperas (Waits) : Reducidas

CONSULTA ORIGINAL (SIN GOVERNOR)

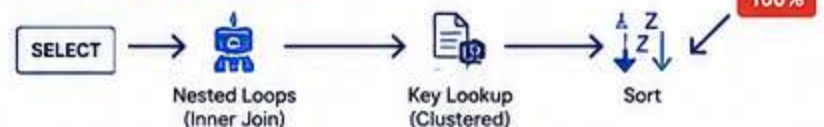
```
SELECT d.IdDiploma, d.FechaEmision, e.NroResolucion
FROM Documento.Diploma d
INNER JOIN Estudiante e
ON d.IdEstudiante = e.IdEstudiante
WHERE d.Estado = 'PENDIENTE';
```

CONSULTA OPTIMIZADA (CON GOVERNOR)

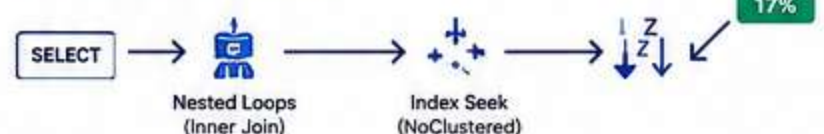
```
WITH DiplomasActivos AS (
SELECT d.IdDiploma
FROM Documento.Diploma d
WHERE d.Estado = 'PENDIENTE'
)
SELECT da.IdDiploma, d.FechaEmision, e.NroResolucion
FROM DiplomasActivos da
JOIN Documento.Diploma d ON da.IdDiploma = d.IdDiploma
JOIN Estudiante e ON d.IdEstudiante = e.IdEstudiante;
```

PLAN DE EJECUCIÓN

PLAN ANTES (SIN GOVERNOR)



PLAN DESPUÉS (CON GOVERNOR)



MEJORA OBTENIDA: 83% menos tiempo de emisión

10 CONCLUSIÓN

Resource Governor permite administrar los recursos del servidor de forma inteligente, priorizando los procesos críticos como la emisión de diplomas en GradosTitulos.

Administrar recursos es administrar prioridades.



11 FRASE PARA RECORDAR

“Administrar recursos es administrar prioridades.”



HERRAMIENTAS ÚTILES

- SSMS (Query Plan)
- Resource Governor
- Activity Monitor
- Extended Events
- DMV (sys.dm_resource_governor_*)



DMV ÚTILES

- sys.dm_resource_governor_resource_pools
- sys.dm_resource_governor_workload_groups
- sys.dm_exec_sessions
- sys.dm_exec_requests



RECURSOS CONTROLABLES

- CPU (MAX_CPU_PERCENT)
- MEMORIA (MAX_MEMORY_PERCENT)

PROBLEMAS COMUNES

- Límite muy bajo = lentitud
- Límite muy alto = sin efecto
- Configuración incorrecta de reglas



RECOMENDACIÓN DBA

Define grupos claros, monitorea constantemente y ajusta según la carga real del sistema.

